

GamutX

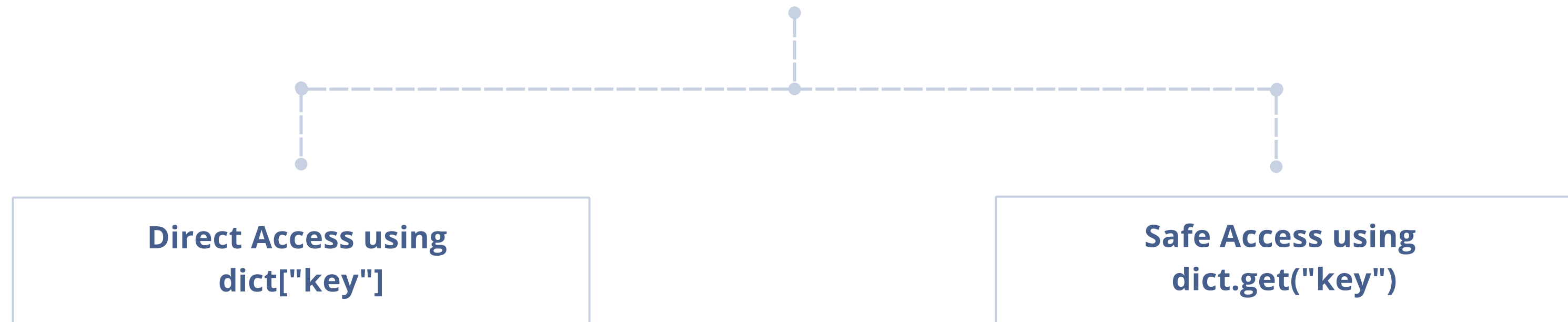
Code Essentials

PYTHON SCRIPTING

Dict - get() Method

- Problem Scenario
- What is get()?
- Using Default Value
- Accessing Values - Using get()
- Direct Access ([]) vs. .get() Method
- Why Use .get()?

When working with dictionaries,
there are **two common ways** to **access values by key**



Problem Scenario

- Imagine you're working on a **VFX pipeline tool** maybe a script that loads asset metadata from a **database** or a **JSON file**.
- To retrieve a value by key, we usually **use square brackets**

```
test.py > ...  
1  asset = {  
2      "name": "Tree_01",  
3      "type": "Model",  
4  }  
5  
6  print("Category:", asset["category"])  
7
```

access a key that may not exist.



But this **raises an error** if the key doesn't exist.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  
  
⊗ PS C:\Users\pralhad\Desktop\test> & C:/Users/pralhad/AppData/Local/Programs/Python/Python39-64/Python.exe test.py  
Traceback (most recent call last):  
  File "c:\Users\pralhad\Desktop\test\test.py", line 8, in <module>  
    print("Category:", asset["category"])  
KeyError: 'category'  
○ PS C:\Users\pralhad\Desktop\test> █
```

Python crashes when key is missing.

Why Is This a Problem?

- In real VFX production, data is often incomplete — especially in early stages.
- Fields like "category", "subdivision_level", or "shader_group" may be:
 - Optional
 - Missing due to artist oversight
 - Not yet set by a pipeline step

So, What's the Solution?

To prevent errors like **KeyError** when accessing a missing key in a dictionary:

- ✓ Python provides a built-in method called **get()**



What is get()?

- The `.get()` method is a **safe** and **reliable** way to **access values from a dictionary**.
- Useful where metadata may be incomplete or inconsistent.

Syntax

`dictionary.get (key, default_value)`

key we want to access in the dictionary

(Optional) Value returned if the key doesn't exist (defaults to None)

Let's Use get() with the Same Example

```
asset = {  
    "name": "Tree_01",  
    "type": "Model",  
}
```

Category Key is missing

```
category = asset.get("category")
```

Instead of error, it returns None

```
print("Category:", category)  
# Category: None
```

📌 Key Points

- Returns the value for the key if it **exists**.
- Returns **None** if the **key doesn't exist** unless you specify a default value.
- Prevents **KeyError**

Using Default Value

- When a dictionary key is missing, `.get()` normally returns `None`.
- But in many cases returning `None` might not be helpful.
- We might want to show a placeholder value, a fallback setting, or a message instead.

◆ Without Default Value

```
person = {"name": "Alice", "age": 25, "city": "London"}  
  
country = person.get("country")  
  
print(country) # Output: None
```

→ "country" is missing, so `.get()` returns **None**.

◆ With Default Value

```
person = {"name": "Alice", "age": 25, "city": "London"}  
  
country = person.get("country", "Not Specified")  
  
print(country) # Output: Not Specified → Default Value
```

Provide a fallback value

Note:

- If the **key exists** → returns the **actual value**.
- If the **key is missing** → returns **default_value** (instead of `None`).

Accessing Values - Using get()

- The **get()** method is a safer way to access elements because it doesn't **raise an error** if the key is not found.
- Instead, it returns None (or a default value you specify).

Accessing an Existing Key

```
person = {"name": "Alice", "age": 25, "city": "London"}  
name = person.get("name")  
print(name)
```



Output: Alice

.get("name") returns "Alice" — just like using `person["name"]`, but safely.

Accessing a Non-Existent Key

```
person = {"name": "Alice", "age": 25, "city": "London"}  
country = person.get("country")  
print(country)
```



Output: None

Instead of raising a **KeyError**, **.get("country")** returns None.

Direct Access ([]) vs. .get() Method

Code Comparison

✗ Using [] (Direct Access)

```
data = {"name": "Alice"}  
  
print(data["age"]) # KeyError
```

Missing key → get **KeyError**

✓ Using .get() (Safe Access)

```
data = {"name": "Alice"}  
  
print(data.get("age")) # Output: None  
  
print(data.get("age", "Not Provided")) # Output: Not Provided
```

default value

Missing key → returns **None**

✗ Direct Access ([])	✓ Safe Access (.get())
Raises a KeyError if the key is missing	Returns None if the key is missing
Can crash script unexpectedly	Keeps script safe and running
Requires extra checks like → <i>if key in dict</i>	Optionally allows a default fallback value
Less flexible for optional or dynamic data	Designed for flexible and safe data access

Why Use .get()?

GamutX

Advantage

-  **Prevents crashes**
-  **Allows default values**
-  **Cleaner code**
-  **Real-world safe**

Description

- Avoids **KeyError** when key is missing
- Returns a fallback when data is missing
- Reduces the need for **if** checks or try-except blocks
- Ideal for working with optional fields in metadata or config

Comparison Table

Feature	<code>dict["key"]</code>	<code>dict.get("key")</code>
• If key exists	✓ Returns value	✓ Returns value
• If key is missing	✗ Raises <i>KeyError</i>	✓ Returns None (or default)
• Custom fallback supported?	✗ No	✓ Yes (<code>dict.get(key, default)</code>)
• Risk of crashing the script	⚠ Yes	✓ No
• Needs extra check?	✓ if key in dict:	✗ Not required
• Recommended for optional data?	✗ No	✓ Yes

Understanding if `__name__ == '__main__'`

- `__name__`
- Using `__name__` with os Module
- Adding if `__name__ == '__main__'` Block
- What is if `__name__ == '__main__'`?
- Real-World Use Case
- Why Its Important

__name__

- `__name__` is a **special built-in variable** in Python.
- It is **automatically defined by Python** when any script or module runs.
- It helps Python know whether:
 - A file is being **run directly**
 - Or being **imported as a module**

Run File Directly

sample.py

```
1 print("__name__ is:", __name__)
2
3 # Output : __name__ is: __main__
4
```

special built-in variable

special value

✓ When we **run the script**, Python sets `__name__` to `"__main__"`. which is a **special value**

✓ This means the file is being run directly.

Import the File

sample.py

```
1 print("__name__ is:", __name__)  
2  
3 # Output : __name__ is: __main__  
4
```



test_import.py

```
1 import sample  
2  
3 # output: __name__ is: sample  
4
```

We import sample module

Python labels imported file by module name

Why Does This Happen?

- When Python imports a file, it treats that file as a module.
- So Python sets: **name = "sample"**

This is the **name** Python uses to refer to the **imported file** as a **module**.

📌 Key Points

- If the file is run directly → **__name__ = "__main__"**
- If the file is imported → **__name__ = "module_name"**

Using `__name__` with `os` Module

- Now that we've seen how `__name__` behaves in **our own scripts**,
- let's look at **how** it works **with an imported module** like `os`.

Example

```
test_import.py
1  import os
2
3  print(os.__name__) # Output: os
```

shows the module's internal name

Why Is This Useful?

It confirms that the `os` module is being **used as an import**, not executed as a **standalone script**.

So far, We've learned that
Python sets the **`__name__`** variable based on how a file is used:

`"__main__"`

→ when a file is **run directly**

```
1 print("__name__ is:", __name__)
```

```
2
```

```
3 # Output : __name__ is: __main__
```

```
4
```

`"module_name"`

→ when a file is **imported**

```
1 import sample
```

```
2
```

```
3 # output: __name__ is: sample
```

```
4
```



How do we use this behavior in real scripts?

You might have come across this line in **Python files**:

Example 1

```
import os, time

def clean_logs(folder, days=7):
    cutoff = time.time() - days * 86400
    for f in os.listdir(folder):
        path = os.path.join(folder, f)
        if f.endswith(".log") and os.path.getmtime(path) < cutoff:
            os.remove(path)
    print("✓ Old logs cleaned up.")

if __name__ == "__main__":
    clean_logs("logs", days=3)
```

Example 2

```
def calc_duration(frames, fps=24):
    return round(frames / fps, 2)

if __name__ == '__main__':
    with open('shots.txt') as f:
        for line in f:
            name, frames = line.strip().split(',')
            dur = calc_duration(int(frames))
            print(f'{name}: {dur}s')
```

It shows up in **many scripts**, especially in:

- Utility scripts
- Testing files
- Command-line tools
- Reusable modules

```
def calc_duration(frames, fps=24):  
    return round(frames / fps, 2)  
  
if __name__ == '__main__':  
    with open('shots.txt') as f:  
        for line in f:  
            name, frames = line.strip().split(',')  
            dur = calc_duration(int(frames))  
            print(f'{name}: {dur}s')
```

But what does it actually mean?

It's a conditional check that asks:

“Is this the file the user is **running directly**?”

-  If **yes** → **Run the code** in the block
-  If **not** (i.e., it's being imported) → **Skip the block**

Adding if __name__ == "__main__" Block

test.py

```
def greet():  
    print("Hello, welcome to the program!")  
  
if __name__ == "__main__":  
    print("Running test.py directly!")  
    greet()
```

runs only when script is main

```
# Running test.py directly!  
# Hello, welcome to the program!
```

checks if file is run directly

What If You Import test.py?

Now create another file **main.py** and run it

```
1 import test
```

-----> *import and run*

Output: (Nothing from *test.py*)

Why No Output?

- Python is importing test.py, not running it directly.
- So test.__name__ becomes "test" — the module name.
- That means:
 - `if __name__ == "__main__"`
 - is False, and the block is skipped.

What is `if __name__ == "__main__":`?

It's Just a Conditional Statement

`if __name__ == "__main__":`



string literal

string value (built-in)

- line is simply an **if condition** — like any other **if** in Python.
- It checks whether the built-in variable `__name__` is equal to the string `"__main__"`.

Why We Write `if __name__ == "__main__"`

- We use this condition when we want to **run specific code** → when the file is **executed directly**,
- when it's **imported** from another file → It **does not run** that specific part of code.

Real-World Use Case

A Weather Application

We're building a weather application. It has:

- A module to fetch data → `data_fetcher.py`
- A module to parse the data → `data_parser.py`
- A main file to run everything → `main.py`

This kind of structure is very common in production **scripts** and **VFX pipelines** where you break tasks into separate files.

File 1: `data_fetcher.py`

```
def fetch_weather_data():  
    print("Fetching weather data...")  
  
if __name__ == "__main__":  
    print("This is data_fetcher.py")  
    fetch_weather_data()  
  
# Output:  
# This is data_fetcher.py  
# Fetching weather data...
```

runs only when this file executes directly

File 2: `data_parser.py`

```
def parse_weather_data(data):  
    print(f"Parsing weather data: {data}")  
  
if __name__ == "__main__":  
    print("This is data_parser.py")  
  
# Output: This is data_parser.py
```

runs only when this file
executes directly

- We can test this **module alone**.
- It **doesn't** run anything **automatically** when imported into **another script**.

Each file can be **tested on its own**; the `__main__` block ensures code **runs only when executed, not when imported**.

File 3: **main.py**

```
main.py / ...
import data_fetcher
import data_parser

def main():
    print("Starting the Weather Application...")

    data = data_fetcher.fetch_weather_data()
    data_parser.parse_weather_data(data)

if __name__ == "__main__":
    main()

# Output:
# Starting the Weather Application...
# Fetching weather data...
# Parsing weather data: None
```

calls fetch function from module

sends data to parser function

execute only when run directly

- **main.py** imports both modules.
- Calls their functions inside a clearly defined **main()** function.
- Uses its own **if __name__ == "__main__"** block to start execution.

key Point

- Even though `data_fetcher.py` and `data_parser.py` both have code inside a `__main__` block, that code:
 -  Runs if we execute those files directly (for testing or debugging)
 -  Does not run when we import them into `main.py`
- This keeps your logic separated and prevents unexpected behavior when building larger systems.

Why Its Important

Benefit

- **Test modules directly**
- **Keep imports clean**
- **Separate logic from reuse**
- **Maintain pipeline safety**

Why It Matters

- You can run a single file to test its logic without affecting others
- Prevents functions from auto-running when imported
- Helps distinguish between reusable code and script behavior
- Avoids unexpected behavior when sharing scripts across tools/pipeline

Python Naming Conventions

- What is a Naming Convention?
- Why Is This Important?
- Pascal Case (UpperCamelCase)
- Snake Case (underscore_case)
- Screaming Snake Case (ALL_CAPS)
- Camel Case (lowerCamelCase)
- Kebab Case (kebab-case)
- Hungarian Notation
- Best Practices
- Benefits of Naming Conventions

What is a Naming Convention?

- A naming convention is a **set of guidelines that developers follow when naming parts of their code**
- To keep it consistent, readable, and understandable by others.

Key Elements That Use Naming Conventions

- **Variables** → `age, file_path, frame_count`
- **Functions** → `calculate_total(), load_asset()`
- **Classes** → `CharacterRig, AssetManager`
- **Constants** → `MAX_RETRIES, DEFAULT_COLOR`
- **File names, etc.** → `texture_loader.py, shot_list.json`

Why Is This Important?

Let's look at a naming example:

```
framecounter = 100  
FrameCounter = 100  
frame_counter = 100  
frameCounter = 100
```

Are these the same variable?

Problems This Can Cause

- **Misunderstandings** between developers
- **Hard-to-trace bugs** caused by typos or mismatched names
- **Messy and inconsistent code**, especially across a team

✓Solution

Python uses **standard naming conventions** to avoid confusion.

- Following these conventions helps us write:
 - **Organized**
 - **Maintainable**
 - **Professional-looking code**
- 

List of naming conventions

- Pascal Case (UpperCamelCase)
 - Camel Case (lowerCamelCase)
 - Snake Case (underscore_case)
 - Screaming Snake Case (ALL_CAPS)
 - Kebab Case (kebab-case)
 - Hungarian Notation
- 

Pascal Case (UpperCamelCase)

What is it?

- Pascal Case means that **each word starts with a capital letter**, including the first one.

How it is Written

```
class CharacterRig:  
    pass  
class AnimationManager:  
    pass
```

💡 VFX Use Case:

PascalCase

```
class TextureLoader:  
    def load_texture(self, file_path):  
        print(f>Loading texture from {file_path}")
```

TextureLoader is a class

Why do we use it?

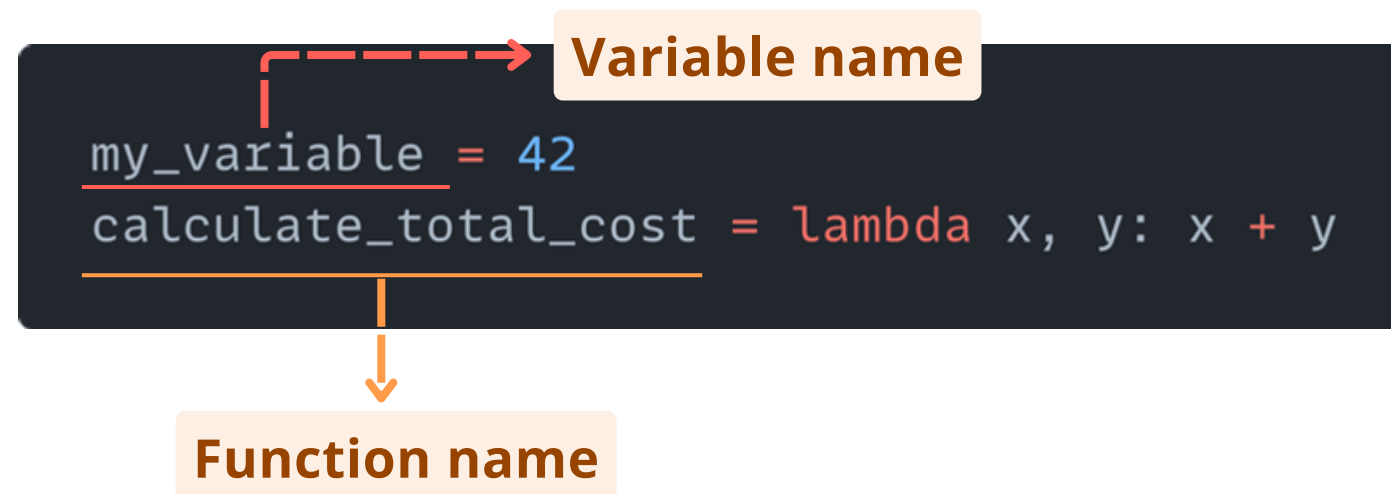
- Used for **class names**.
- Helps distinguish classes from variables and functions at a glance.

Snake Case (underscore_case)

What is it?

- Snake Case uses **underscores to separate words**, with all **letters in lowercase**.
- Where do we use it? → **Variable names and Function names**

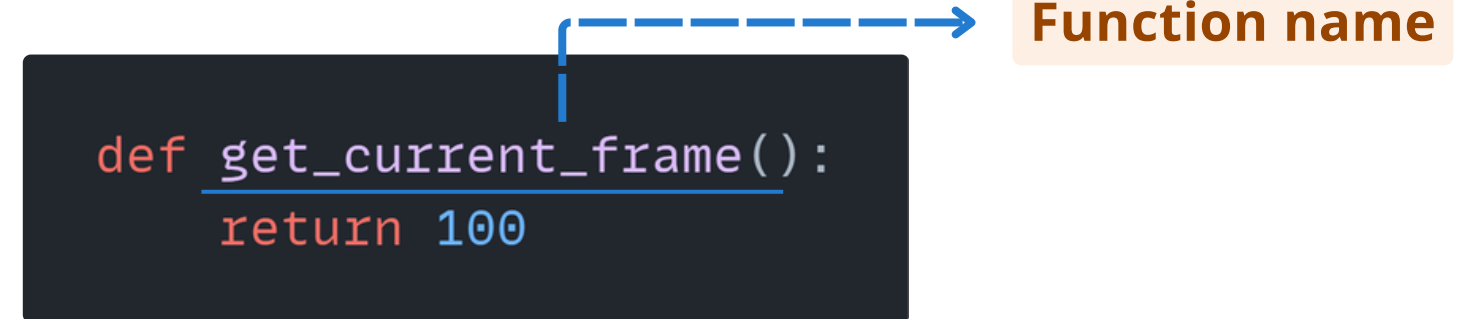
How to use it



The diagram shows two lines of code in a dark box. The first line is `my_variable = 42`, with a red dashed arrow pointing from `my_variable` to an orange box labeled "Variable name". The second line is `calculate_total_cost = lambda x, y: x + y`, with an orange arrow pointing from `calculate_total_cost` to an orange box labeled "Function name".

```
my_variable = 42
calculate_total_cost = lambda x, y: x + y
```

💡 VFX Use Case:



The diagram shows a function definition in a dark box: `def get_current_frame():` followed by `return 100`. A blue dashed arrow points from `get_current_frame` to an orange box labeled "Function name".

```
def get_current_frame():
    return 100
```

This follows PEP 8 — Python's official style guide.

Why do we use it?

- Officially **recommended** in Python's PEP 8
- Highly **readable, especially for long names**
- Widely used in scripting environments like VFX pipelines

Screaming Snake Case (ALL_CAPS)

What is it?

- A variant of Snake Case where **all letters are uppercase**.
- Where do we use it? → **CONSTANTS** → fixed values that should not change

How to use it

```
MAX_RETRY_COUNT = 5  
PI = 3.14159
```

→ **CONSTANT**

💡 VFX Use Case:

```
FRAME_RATE = 24  
TEXTURE_FORMAT = "EXR"
```

Use this to signal that a value is **constant** and should not be changed.

Why do we use it?

- Makes constants easy to identify.
- Signals that a value should not be modified.

Camel Case (lowerCamelCase)

What is it?

Similar to Pascal Case, but the **first word starts** with a **lowercase** letter. then rest start with Capital

Why is it not preferred?

Python follows **Snake Case** for variables and functions instead.

✗ Avoid this in Python:

```
def getFrameRate():  
    return 24
```

✓ Use Snake Case instead:

```
def get_frame_rate():  
    return 24
```

Snake Case

Where is it used?

- ✗ Not recommended in Python.
- ✓ Used in JavaScript, Java, and other languages for variable and function names.

Kebab Case (kebab-case)

What is it?

- Words are separated by **hyphens (-)** instead of **underscores or capitalization**.
- **✗ Not used in Python code** — hyphens are treated as the minus operator.
- **Where do we use it?** → File names, URLs, web development

How is it written

```
my-variable.txt  
calculate-total-cost.sh
```

💡 VFX Use Case:

```
textures/character-texture.exr
```

✓ Fine for naming **files**, not for **Python variables or functions**.

Hungarian Notation

What is it?

- Prefixes variables with **type indicators** (like `int`, `str`, `lst`, etc.).
- **✗ Outdated** — not used in modern Python code.

Example (avoid this):

```
strUserName = "JohnDoe"  
intCounter = 5  
lstItems = [1, 2, 3]
```

→ unclear names

→ Use descriptive, Pythonic names instead:

```
username = "JohnDoe"  
counter = 5  
items = [1, 2, 3]
```

✓ Python emphasizes **descriptive names**, not type prefixes.

Best Practices

- ✓ Use **Pascal Case** for classes
- ✓ Use **Snake Case** for functions and variables
- ✓ Use **Screaming Snake Case** for constants
- ✗ Avoid **Camel Case**, **Kebab Case**, and **Hungarian Notation** in Python code

Naming Style	Used in Python?	Common Use	Example
• Pascal Case	→ ✓ Yes	→ Class names	→ <code>MyClass, AssetManager</code>
• Snake Case	→ ✓ Yes (PEP 8 standard)	→ Variables, functions	→ <code>my_variable, get_frame()</code>
• Screaming Snake Case	→ ✓ Yes	→ Constants	→ <code>MAX_SIZE, DEFAULT_PATH</code>
• Camel Case	→ ✗ Not preferred	→ Used in JS/Java	→ <code>myVariable, doSomething()</code>
• Kebab Case	→ ✗ Not preferred	→ File names, URLs	→ <code>file-name.txt</code>
• Hungarian Notation	→ ✗ Not preferred	→ Old C-style codebases	→ <code>strName, intCount</code>

Benefits of Naming Conventions

Benefit

Explanation

- | | |
|---------------------------------|--|
| • Improves Readability | → Clear names help others quickly understand what a variable, function, or class does. |
| • Ensures Consistency | → Everyone on the team follows the same style, reducing confusion. |
| • Prevents Bugs | → Consistent names reduce errors caused by typos or mixing styles. |
| • Supports Collaboration | → Easier for teams to work on shared code without needing to decipher naming logic. |
| • Professional Standards | → Aligns with best practices used across open-source projects and Python libraries. |

Relative Path vs Script Relative Path

- What is a Relative Path?
- What is a Script-Relative Path?
- Building File Paths
- Handling weather.json
- Reading weather.json Dynamically
- Why This Is Important

What is a Relative Path?

Relative path **refers to a file location based on the current working directory**, not the full system path.

Folder Structure

```
/project
├── test.py
└── weather.json
```

weather.json

```
{
  "city": "San Francisco",
  "temperature": 18,
  "unit": "C",
  "forecast": [
    {"day": "Mon", "cond": "Sunny"},
    {"day": "Tue", "cond": "Cloudy"},
    {"day": "Wed", "cond": "Rain"}
  ]
}
```

Code:

```
import json

with open("weather.json", 'r') as file:
    data = json.load(file)

print(data)
```

Relative path (points to "weather.json")

opens the JSON (points to the `open()` function)

```
# {'city': 'San Francisco', 'temperature': 18, 'unit': 'C',
'forecast': [{'day': 'Mon', 'cond': 'Sunny'}, {'day': 'Tue', 'cond':
'Cloudy'}, {'day': 'Wed', 'cond': 'Rain'}]}
```

- Since **test.py** and **weather.json** are in the same folder, just using **"weather.json"** works.
- The file path is **relative to where the script is executed**.

What If We Move test.py?

Updated Folder Structure

```
/project
├── /scripts
│   └── test.py
└── weather.json
```

Updated path in **test.py**:

```
import json

with open("../weather.json", "r") as file:
    data = json.load(file)

print(data)
```

Relative path

```
# {'city': 'San Francisco', 'temperature': 18, 'unit': 'C', 'forecast':
[{'day': 'Mon', 'cond': 'Sunny'}, {'day': 'Tue', 'cond': 'Cloudy'},
{'day': 'Wed', 'cond': 'Rain'}]}
```

- `../weather.json` → tells Python to look one folder up
- `"../"` → means "start from current directory"

Now let's Move Weather.json file inside another folder

What If weather.json Moves to data/?

Let's say we move **weather.json** into a new folder called **data/** inside your **project** directory.

Updated Folder Structure

```
/project
├── /scripts
│   └── test.py
├── /data
│   └── weather.json
```

Now, the relative path from **test.py** would be:

```
import json
with open("./data/weather.json", 'r') as file:
    data_dict = json.load(file)

print(data_dict)
```

Relative path

```
# {'city': 'San Francisco', 'temperature': 18, 'unit': 'C', 'forecast':
# [{'day': 'Mon', 'cond': 'Sunny'}, {'day': 'Tue', 'cond': 'Cloudy'},
# {'day': 'Wed', 'cond': 'Rain'}]}
```

path for weather.json

./data/weather.json

folder path relative to script's location

✓ So, relative paths are not fixed — they **change** based on **where we run the script from**.

Why This Can Be a Problem

In **production** (especially in VFX pipelines):

- Scripts are **run from different tools, launchers, or shells**
- Execution **folders vary** depending on **how a task is triggered**
- **Relative paths can break if the script is executed from another place**

| To Solve this we **Use Script Relative Path**



What is a Script-Relative Path?

A **script-relative path** is **calculated** based on the **location of the script file itself**, not the folder you're running the script from.

This means:

- No matter where we **run the script** from — **terminal, scheduler, IDE**, another file
- the **path is calculated based on the actual script file's location**.

Building File Paths

Exploring os.path & os.name

```
import os
print(os.path)
# <module 'ntpath' from 'C:\\Users\\pralhad\\AppData\\Local\\Programs\\Python\\Python39\\lib\\ntpath.py'>

print(os.name)
# nt
```

Annotations:

- Orange dashed arrow from `import os` to **Show the path of module**
- Blue dashed arrow from `print(os.name)` to **Identify OS Type**
- Red dashed arrow from the path string in the first output to **Absolute Path**

→ `os.name` shows our **OS type**:

- **'nt'** for **Windows**
- **'posix'** for **macOS/Linux**

Using `__file__`

- `__file__` is a **built-in variable**.
- It contains the **full path** of the script **being executed**.
- This is useful for locating files that live **in the same folder** as your script — like **config files**, **assets**, or **data files**.

```
import os
print(__file__)

# c:\Users\pralhad\Desktop\test\scripts\test.py
```

Show current script's pathCurrent file absolute path

Using os.path.dirname()

- This returns **only the directory path**, without the **filename**.
- **Useful** when our **script needs to access other files in the same folder**.

```
import os
print(os.path.dirname(__file__))
# c:\Users\pralhad\Desktop\test\scripts
```

extracts only the folder path

Store current script's file path

Using `os.path.abspath()`

Now, let's retrieve the **absolute path of the current script** using `os.path.abspath()`.

```
import os
print(os.path.abspath(__file__))
# c:\Users\pralhad\Desktop\test\scripts\test.py
```

Diagram annotations:

- `os.path.abspath()` returns the absolute (full) path of a file
- `__file__` Store current script's filename
- `c:\Users\pralhad\Desktop\test\scripts\test.py` complete script path

It **works** regardless of **where the script is executed from**.

Combining os.path.dirname() & os.path.abspath()

To get the **absolute directory** of the script (not including the filename):

```
import os
script_dir = os.path.dirname(os.path.abspath(__file__))
print(script_dir)

# Output: C:\project\scripts
```

Diagram annotations:

- Annotation 1: **removes filename, keeps folder path** (points to `os.path.dirname()`)
- Annotation 2: **gets full path including filename** (points to `os.path.abspath(__file__)`)

- `os.path.abspath(__file__)` → gets full path including filename
- `os.path.dirname()` → removes filename, keeps folder path

This is the **recommended approach** for **script-relative paths** in Python.

Handling weather.json

Calculating the Path to weather.json

We will **dynamically calculate** the path to **weather.json**.

```
import os
cur_file = os.path.dirname(os.path.abspath(__file__))
weather_json_file_path = cur_file + "weather.json"
print(weather_json_file_path)
# c:\Users\pralhad\Desktop\test\scriptweather.json
```

gets the directory path

full path of the current script

concatenates folder path and filename

missing slash causes incorrect path

Folder Structure

```
/project
├── test.py
└── weather.json
```

- This **fails** because it **misses the slash** (/ or \) between the **folder and file name**.
- **Concatenating** paths manually is risky and **error-prone across platforms**.

Use os.path.join()

let's use **os.path.join()**, which ensures the **correct path separator is used** for the **operating system**.

```
import os

cur_file = os.path.dirname(os.path.abspath(__file__))

weather_json_file_path = os.path.join(cur_file, "weather.json")

print(weather_json_file_path)
# c:\Users\pralhad\Desktop\test\scripts\weather.json
```

Diagram illustrating the path construction:

- `os.path.join()` combines `cur_file` and `"weather.json"` into the final path.
- The final path is `c:\Users\pralhad\Desktop\test\scripts\weather.json`.

os.path.join() automatically handles slashes for **Windows (\)** and **macOS/Linux (/)**.

What If `weather.json` Moves inside `data` Folder?

```
/project
├── test.py
└── data
    └── weather.json
```

Now, `weather.json` is inside a `data/` folder.

Adjusting the Path for Subfolders

```
import os

cur_file = os.path.dirname(os.path.abspath(__file__))

weather_json_file_path = os.path.join(cur_file, "data", "weather.json")

print(weather_json_file_path)
# c:\Users\pralhad\Desktop\test\data\weather.json
```

base directory of the current script

subfolder name

target file inside that subfolder

- We just added **"data"** as a folder level in **os.path.join()**.
- This keeps the **path flexible and accurate** even if your structure changes

Reading weather.json Dynamically

Now that we have the correct path, let's read the contents of **weather.json**.

```
import os
```

Extracts just the folder of the script

```
script_dir = os.path.dirname(os.path.abspath(__file__))
```

Gets the absolute path to the current script

```
weather_json_file_path = os.path.join(script_dir, "data", "weather.json")
```

builds a full path to the weather.json file

```
with open(weather_json_file_path, 'r') as file:  
    data = file.read()  
    print(data)
```

```
...
```

```
# Output:
```

```
{  
  "city": "San Francisco",  
  "temperature": 18,  
  "unit": "C",  
  "forecast": [  
    {"day": "Mon", "cond": "Sunny"},  
    {"day": "Tue", "cond": "Cloudy"},  
    {"day": "Wed", "cond": "Rain"}  
  ]  
}
```

```
...
```

Folder Structure

```
/project  
├── test.py  
└── data  
    └── weather.json
```

Why This Is Important

This method makes our code:

- **Robust** → it works no matter where we run the script from.
- **Portable** → easy to share across systems or with teammates.
- **Flexible** → works even if we move our script or the weather.json file.

We **don't need to hardcode full paths** like: `"C:/Users/Name/Documents/Project/data/weather.json"`

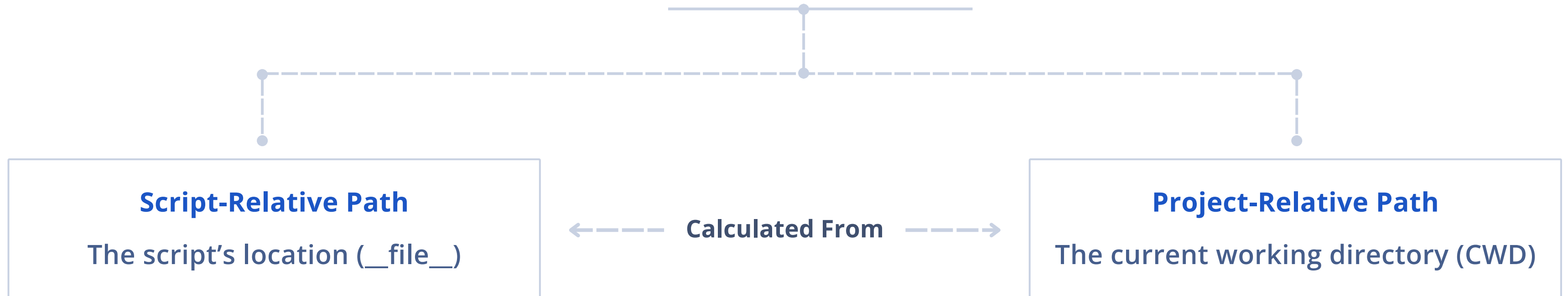
Instead, let our script figure it out **dynamically using `__file__` and `os.path`**.

Script-Relative Path vs Project Relative Path

- Script-Relative Path vs Project Relative Path
- Script-Relative Path
- Project Relative Path
- Script-Relative Path vs Project Relative Path
- Situational Recommendations

Script-Relative Path vs Project Relative Path

When working with file paths in Python, you have **two main** approaches:



Script-Relative Path

- A **script-relative path** is calculated based on **where the script file is physically located**, not from where it's run.
- This makes your code **portable**, **reliable**, and **safe** to use in any environment

Example Scenario

Folder Structure

```
project/  
├── script.py  
└── sample.json
```

both files are in the same folder.

sample.json

```
{  
  "project": "VFX Training",  
  "version": 1,  
  "enabled": true,  
  "tools": ["loader", "exporter", "viewer"]  
}
```

script.py

```
import json  
  
with open("sample.json", "r") as file:  
    data = json.load(file)  
  
print(data)  
  
# {'project': 'VFX Training', 'version': 1, 'enabled':  
  True, 'tools': ['loader', 'exporter', 'viewer']}
```

works because Python looks for **sample.json** in the **current working directory**

Move script.py to a New Location

Old Folder Structure

project/
├── script.py
└── sample.json

Updated Folder Structure

project/
├── data/
│ └── sample.json
└── tools/
 └── script.py

script.py

```
import json

with open("sample.json", "r") as file:
    data = json.load(file)

print(data)

# File "c:\Users\pralhad\Desktop\project\tools\script.py", line 11, in <module>
#     with open(file_path, 'r') as file:
# FileNotFoundError: [Errno 2] No such file or directory:
# 'c:\\Users\\pralhad\\Desktop\\project\\tools\\sample.json'
```

Python search file inside tool Folder

✗ Not in Data Folder

Since the **script** is now in a **different Folder**,
we need to **dynamically construct the path** to **sample.json**.

For that
↓

We use **Script Relative Path**



Using Script-Relative Path

To make this work reliably, we **dynamically construct the path** to **sample.json** using the script's location.

```
script.py / ...
import os
import json

script_dir = os.path.dirname(os.path.abspath(__file__))

file_path = os.path.join(script_dir, "..", "data", "sample.json")

file_path = os.path.abspath(file_path)

with open(file_path, "r") as file:
    data = json.load(file)

print(data)
# {'project': 'VFX Training', 'version': 1, 'enabled': True, 'tools':
# ['loader', 'exporter', 'viewer']}
```

Annotations:

- current script's file path
- Starting folder (where the script lives)
- Moves one level up
- Target file

Updated Folder Structure

```
graph TD
    project[project/] --> data[data/]
    project --> tools[tools/]
    data --> sample[sample.json]
    tools --> script[script.py]
```

By going up one level (..) and into **data/**, we correctly reach the JSON file.

Use Cases

Portability:

- The **script** will always find its **resources**, no matter where it is **run from**.

Deployment Flexibility:

- Useful for scripts that may be deployed on **different machines**, scheduled as **cron jobs**, or run in **unpredictable environments**.

Larger Projects:

- Reduces "**file not found**" errors in projects where multiple **scripts share resources**.

Project Relative Path

- A **Project-Relative Path** (also called **Working Directory Relative Path**) refers to file paths that are resolved **relative to the folder you're in when you run the script**, not where the script is saved.
- That means it assumes we are running the script from the folder that also contains the target file.

Example Scenario

Folder Structure

📁 project/
├── script.py
└── sample.json

script.py

```
import json

file_path = "sample.json"

with open(file_path, 'r') as file:
    data = json.load(file)

print(data)
# {'project': 'VFX Training', 'version': 1, 'enabled': True, 'tools':
# ['loader', 'exporter', 'viewer']}
```

Relative path (project-relative)

- works **only** if the **script is run from the project/ folder**
- Python will look for sample.json in the **current working directory**,
- **not in the script's folder.**

How Path Resolution Works

Where we run the script from

project/

script.py → Run from **root folder**

sample.json

Python Looks For File In

project/

script.py

sample.json

```
import json

file_path = "sample.json"

with open(file_path, 'r') as file:
    data = json.load(file)

print(data)
# {'project': 'VFX Training', 'version': 1, 'enabled': True, 'tools':
# ['loader', 'exporter', 'viewer']}
```

project/

sample.json

tools/

script.py → Run from **Tools folder**

Python Looks For File In

project/

sample.json

tools/

script.py

```
import json

with open("sample.json", "r") as file:
    data = json.load(file)

print(data)

# File "c:\Users\pralhad\Desktop\project\tools\script.py", line 11, in <module>
#   with open(file_path, 'r') as file:
# FileNotFoundError: [Errno 2] No such file or directory:
# 'c:\\Users\\pralhad\\Desktop\\project\\tools\\sample.json'
```

Where we run the script from**Python Looks For File In****Will It Work?**

- **/home/user/project/**

→ /home/user/project/sample.json

→  Yes

- **/home/user/project/tools/**

→ /home/user/project/tools/sample.json

→  No

The script will only work if it's **run from the correct directory** — otherwise, it will raise a **FileNotFoundError**.

Limitations

- Using project-relative paths (e.g. "**sample.json**"):
 - ✗ Breaks if you run the script from the wrong directory
 - ✗ Not reliable in automated tools (e.g. **cron jobs**, **render farm scripts**)
 - ✗ **Not reusable** when moved across folders or projects
- This approach only works when:
 - We know **exactly** where the **script will be run from**
 - Our **working directory is always the same** as the file location

Use Cases

Controlled Environments:

- Best suited for **systems** where the working directory is guaranteed to be **consistent**.

Simple Scripts:

- Ideal for **quick utilities** or **single-use scripts** run in a known environment.

Small Projects:

- Useful when scripts and resources are **tightly coupled**, and the working directory is **predictable**.

Script-Relative Path vs Project Relative Path

Feature	Script-Relative Path	Direct Relative Path
• Path Basis	→ Based on the location of the script file (<code>__file__</code>)	→ Based on the current working directory (CWD)
• Portability	→  Highly portable — works regardless of where you run it	→  Less portable — breaks if run from the wrong directory
• Flexibility	→  Great for flexible deployment, automation, sharing	→  Only reliable in controlled or manual environments
• Error-Prone	→  Less error-prone — avoids “file not found” issues	→  More error-prone if CWD isn't what you expect
• Use Case	→ Larger projects, shared scripts, scheduled jobs	→  Small tools, one-off scripts, dev/testing scenarios

Situational Recommendations

Use Script-Relative Path When:

- The script might be executed from **different directories** or **environments**.
- You're deploying the **script** on **multiple machines** with **varying configurations**.
- The script needs to find **resources bundled** with it (e.g., **configuration** files, **data** files).
- **Portability** and **flexibility** are important.

Use Direct Relative Path When:

- The script will **always** be **executed** from a known **directory**.
- It's a quick one-off utility or **test script**.
- You're working in a **controlled environment** (e.g., within a **single project directory**).
- Simplicity is a **priority**, and **deployment concerns** are minimal.

In **Production**,
always use **Script-Relative Paths**.



Create a Function to Calculate File Size in Kilobytes

```
def calculate_file_size_in_kb(file_size_bytes):  
    """Converts file size from bytes to kilobytes (KB)."""  
    return file_size_bytes / 1024 # 1 KB = 1024 Bytes  
  
# Example usage  
file_size_kb = calculate_file_size_in_kb(2048)  
print(f"File size: {file_size_kb} KB") # Output: File size: 2.0 KB
```

Define a Function to Format Shot Names Properly

```
def format_shot_name(sequence_number, shot_number):  
    """Formats shot names as 'SEQ###_SHOT###'."""  
    return f"SEQ{sequence_number:03d}_SHOT{shot_number:03d}"  
  
# Example usage  
shot_name = format_shot_name(12, 5)  
print(shot_name) # Output: SEQ012_SHOT005
```

Thank You

